Design and Implementation of a 6-DOF Stewart Platform on an Embedded Microcontroller with a FreeRTOS Control Architecture

Wm. Sam Schneidereith

This paper presents the design, kinematic analysis, and real-time embedded software implementation of a six-degree-of-freedom (6-DOF) Stewart Platform actuated by six RC hobby servos. The system runs on an ESP32 microcontroller and communicates with a PCA9685 16-channel PWM driver over I^2C to generate servo control signals. A closed-form inverse kinematics (IK) solver computes the required horn angles from a desired platform pose represented as a translation and ZYX Euler rotation, replacing an initial numerical search that was measured to be too slow for a 50 Hz control rate. Smooth trajectories are generated with quintic pose interpolation. The firmware is built on FreeRTOS as three prioritized tasks—a fixed-period control task, a trajectory sequencer, and a reporting task—connected by a single-slot overwrite mailbox. The control loop worst-case execution time is measured at roughly 3.7 ms against a 20 ms period. The layered firmware architecture enables straightforward extension to new geometries and motion profiles.

I. Introduction

A Stewart Platform is a class of parallel manipulator capable of motion in all six degrees of freedom: translation along the x, y, and z axes, and rotation about those same axes (roll, pitch, and yaw) [1]. Unlike serial manipulators, which chain joints end-to-end, a Stewart Platform connects its moving plate (the *platform*) to a fixed base through six independent actuated legs arranged in parallel. This topology provides inherently high structural stiffness, good load distribution across all actuators, and precise positional control.

Parallel manipulators of this class see widespread use in flight simulators, camera stabilization rigs, precision machining fixtures, and haptic interfaces. Implementing a small-scale version using commodity hobby-grade components offers a practical and low-cost testbed for studying robotic kinematics, real-time embedded control, and motion planning algorithms without access to industrial hardware.

This paper is organized as follows. Section II describes the mechanical design and hardware configuration. Section III derives the forward and inverse kinematics, including the closed-form solution and its comparison with a numerical baseline. Section IV presents the software architecture, including the FreeRTOS task design. Section V discusses system performance, limitations, and directions for future work.

II. Mechanical Design and Hardware

A. Platform Geometry

The platform consists of two approximately hexagonal plates—a fixed base and a moving platform—connected by six rigid rods. Each rod attaches at the base end to a servo horn tip and at the platform end to a ball-and-socket joint. The geometry is parameterized by the values listed in Table 1.

The six servo pivot points on the base are arranged in three pairs, each pair straddling a primary symmetry axis. Their azimuth angles are:

$$\theta_{\text{base}} = \left\{ \frac{15\pi}{12}, \frac{17\pi}{12}, \frac{23\pi}{12}, \frac{\pi}{12}, \frac{7\pi}{12}, \frac{9\pi}{12} \right\} \quad \text{[rad]} \tag{1}$$

The platform joints are similarly arranged in offset pairs so that each leg connects a base servo to its nearest platform ball joint, minimizing lateral bending forces on the servo horns.

Table 1 Stewart Platform Geometric Parameters

Parameter	Symbol	Value
Rod length (rod + rod end)	L	190 mm
Horn length	r	30 mm
Base joint radius	R_b	88.9 mm
Base joint height below datum	h_b	−19 mm
Platform joint radius	R_p	74.1 mm
Platform joint height below platform	h_p	−2.5 mm

B. Servo and Driver Hardware

Six RC servo motors serve as actuators. Each servo is connected to one channel of a PCA9685 16-channel PWM driver, which communicates with the ESP32 over I^2C at 400 kHz. The PCA9685 is configured to output a 50 Hz PWM signal, consistent with the standard RC servo control protocol [2].

Each servo has individually calibrated minimum and maximum pulse widths to account for unit-to-unit variation, and an inversion flag to handle the alternating physical orientation of servos around the base ring. The per-servo configuration is listed in Table 2.

Table 2 Per-Servo PWM Calibration and Configuration

Servo	Channel	Min Pulse (µs)	Max Pulse (µs)	Inverted
1	0	450	2400	Yes
2	1	400	2400	No
3	2	430	2400	Yes
4	3	420	2350	No
5	4	550	2490	Yes
6	5	400	2300	No

Every other servo faces the opposite direction around the base ring. The inversion flag causes the software to negate the commanded angle for those channels, so a positive angle always lifts the corresponding horn tip in the same upward sense regardless of physical orientation.

III. Kinematics

A. Coordinate System and Pose Representation

The platform pose is represented as a 6-vector:

$$\mathbf{q} = \begin{bmatrix} x & y & z & \phi & \theta & \psi \end{bmatrix}^T \tag{2}$$

where (x, y, z) is the translation of the platform centroid relative to the base centroid in meters, and (ϕ, θ, ψ) are roll, pitch, and yaw in radians. The rotation matrix is constructed using the ZYX Euler convention:

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_z(\psi)\, \mathbf{R}_y(\theta)\, \mathbf{R}_x(\phi) \tag{3}$$

B. Platform Joint Positions

Given a pose $\mathbf{q}$, the world-frame position of the i-th platform joint is:

$$\mathbf{P}_i = \mathbf{R}\, \mathbf{p}_i + \mathbf{T} \tag{4}$$

where $\mathbf{p}_i$ is the joint position in the platform body frame and $\mathbf{T} = [x,\ y,\ z]^T$.

C. Inverse Kinematics

Given a desired pose, the inverse kinematics (IK) problem is to find the six servo horn angles $\alpha_i \in [-45, +45]$ such that each rod exactly spans the distance between the corresponding horn tip and platform joint.

1. Horn Tip Kinematics

Each servo horn rotates about the servo axle. The tangential unit vector perpendicular to the radial direction in the horizontal plane is:

$$\hat{\mathbf{r}}_i = \frac{1}{\|\mathbf{b}_i^{xy}\|} \begin{bmatrix} b_{ix} \\ b_{iy} \\ 0 \end{bmatrix}, \qquad \hat{\mathbf{t}}_i = \begin{bmatrix} -\hat{r}_{iy} \\ \hat{r}_{ix} \\ 0 \end{bmatrix} \tag{5}$$

For inverted servos $\hat{\mathbf{t}}_i$ is negated. The horn tip position as a function of angle α is then:

$$\mathbf{H}_i(\alpha) = \mathbf{b}_i + \hat{\mathbf{t}}_i \, (r \cos\alpha) + \hat{\mathbf{z}} \, (r \sin\alpha) \tag{6}$$

where $\mathbf{b}_i$ is the servo pivot, r is the horn length, and $\hat{\mathbf{z}} = [0, 0, 1]^T$. The rod constraint for leg i requires $\|\mathbf{P}_i - \mathbf{H}_i(\alpha_i)\| = L$.

2. Closed-Form Solution

Because each horn tip moves in a single plane, the rod constraint reduces to one trigonometric equation per leg. Let $\mathbf{d}_i = \mathbf{P}_i - \mathbf{b}_i$ be the vector from the servo axle to the platform joint. Then

$$\|\mathbf{P}_i - \mathbf{H}_i(\alpha)\|^2 = \|\mathbf{d}_i - r\cos\alpha \, \hat{\mathbf{t}}_i - r \sin\alpha \, \hat{\mathbf{z}}\|^2 = L^2. \tag{7}$$

Since $\hat{\mathbf{t}}_i$ is a unit vector orthogonal to $\hat{\mathbf{z}}$, expanding the left-hand side gives

$$A_i \cos\alpha + B_i \sin\alpha = C_i, \tag{8}$$

with

$$A_i = 2r \, (\mathbf{d}_i \cdot \hat{\mathbf{t}}_i), \quad B_i = 2r \, d_{iz}, \quad C_i = \|\mathbf{d}_i\|^2 + r^2 - L^2. \tag{9}$$

Equation (8) has the closed-form solutions

$$\alpha_i^{\pm} = \operatorname{atan2}(B_i, A_i) \pm \arccos\left(\frac{C_i}{\sqrt{A_i^2 + B_i^2}} \right). \tag{10}$$

Leg i is reachable only if $|C_i| \le \sqrt{A_i^2 + B_i^2}$; otherwise the arccosine argument leaves $[-1, 1]$ and the pose is rejected. A small tolerance is applied before clamping to absorb floating-point rounding. The case $\sqrt{A_i^2 + B_i^2} \approx 0$ is treated as degenerate and also rejected.

The two roots correspond to the two elbow configurations of the horn. Each root is converted to degrees and tested against the servo limits. If both are valid, the solver selects the root closest to the leg's previous commanded angle, which keeps consecutive poses on the same branch and avoids discontinuous jumps. If only one is valid, it is used, and if neither is valid the pose is rejected. The procedure is summarized in Algorithm 1.

3. Numerical Baseline and Timing Comparison

The first implementation used a numerical search instead. For each leg, a uniform sweep over $\alpha \in [-45, +45]$ at 0.1° resolution evaluated the residual

$$e_i(\alpha) = \left| \, \|\mathbf{P}_i - \mathbf{H}_i(\alpha)\| - L \, \right| \tag{11}$$

and selected the angle with the smallest value, accepting the pose only if $e_i < 0.5\,\text{mm}$ for all six legs. Each solve therefore requires several thousand horn-tip evaluations per leg, each involving trigonometric functions and a square root.

Both solvers were benchmarked on the ESP32 by timing 100 consecutive solves of a representative pose (neutral height with 0.2 rad pitch). The results are given in Table 3. The numerical solver averaged about 34 ms per solve, which

Algorithm 1 Closed-Form Inverse Kinematics

Require: Pose $\mathbf{q}$, previous angles $\{\alpha_i^{\text{prev}}\}$, geometry $\{L, r, \mathbf{b}_i, \mathbf{p}_i, \hat{\mathbf{t}}_i\}$, servo limits
Ensure: Servo angles $\{\alpha_i\}$ or FAIL
Compute $\mathbf{R}$ from (ϕ, θ, ψ) via Eq. (3)
for $i = 1$ to 6 **do**
  $\mathbf{P}_i \leftarrow \mathbf{R}\,\mathbf{p}_i + \mathbf{T}$ ▷ Eq. (4)
  $\mathbf{d}_i \leftarrow \mathbf{P}_i - \mathbf{b}_i$
  Compute A_i, B_i, C_i
  if $\sqrt{A_i^2 + B_i^2} \approx 0$ **or** $|C_i| > \sqrt{A_i^2 + B_i^2}$ **then return** FAIL
  end if
  Compute $\alpha_i^{\pm}$ via Eq. (10), in degrees
  Keep roots inside the servo limits
  if no root remains **then return** FAIL
  end if
  $\alpha_i \leftarrow$ remaining root closest to α_i^{prev}
end for
return $\{\alpha_i\}$

exceeds the 20 ms update period and caps the update rate near 29 Hz. The closed-form solver averaged about 71 µs, roughly 480 times faster, so the solver is no longer a meaningful part of the control-loop budget. The closed-form angles were verified to match the numerical solver's angles for identical poses, and the platform was then validated qualitatively on hardware by running the full six-DOF motion sequence.

Table 3 IK Solver Benchmark on the ESP32 (100 solves)

Solver	Mean time per solve	Maximum solve rate
Numerical sweep	34.4 ms	29 Hz
Closed form	71 µs	14 kHz

D. Trajectory Interpolation

To generate smooth motions, the sequencer interpolates between the current pose $\mathbf{q}_0$ and the target $\mathbf{q}_f$ over N steps at the control period $\Delta t = 20$ ms, using a quintic easing function:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \qquad \tau_k = \frac{k}{N}, \tag{12}$$

$$N = \max\Big(1,\ \text{round}\Big(\frac{t_{\text{duration}}}{\Delta t}\Big)\Big), \qquad \mathbf{q}_k = \mathbf{q}_0 + s(\tau_k)\,(\mathbf{q}_f - \mathbf{q}_0). \tag{13}$$

The easing function has zero first and second derivatives at both endpoints, so each move starts and ends with zero velocity and zero acceleration. Because the pose is updated on a fixed Δt grid, requested durations are quantized to whole multiples of the control period. IK is solved at each interpolated pose by the control task. If any pose is unreachable, the solver rejects it and the platform continues to hold its last valid pose.

IV. Software Architecture

A. Layer Overview

The firmware is structured into well-separated layers, summarized in Table 4. The hardware layers are unchanged by the RTOS port: the tasks sit above the platform model and use it through its existing interface.

Table 4 Firmware Layer Breakdown

Layer	Files
Application / startup	`App.cpp`, `main.cpp`
RTOS tasks and configuration	`ControlTask.cpp`, `SequencerTask.cpp`, `ReporterTask.cpp`, `PoseMailbox.cpp`, `rtos_config.h`
Motion definitions	`motion.h`
Platform control	`StewartPlatform.cpp`
Geometry	`StewartGeometry.cpp`, `StewartConfig.h`
Servo abstraction	`Servo.cpp`
PWM driver	`PCA9685.cpp`
I^2C HAL	`i2c.cpp`
Utilities	`linalg.h`, `interpolate.cpp`, `logger.cpp`

B. I^2C and PCA9685 Driver

The lowest layer wraps the Arduino `Wire` library into an `i2c` namespace exposing `writeByte` and `readByte` primitives. The `PCA9685` class uses these to configure the MODE1 register, set the PWM frequency via the prescaler, and write per-channel on/off tick counts. Pulse widths are converted to 12-bit counts by:

$$\text{counts} = \left\lfloor \frac{t_{\mu s} \times 4096}{20000} \right\rfloor \tag{14}$$

assuming the standard 50 Hz (20 ms period) PWM signal.

C. Servo Abstraction

The `Servo` class maps a commanded angle α in degrees to a pulse width using a linear calibration between `minPulse` (-90) and `maxPulse` ($+90$):

$$t = \frac{\alpha + 90}{180}, \qquad \mu s = t_{\min} + (t_{\max} - t_{\min})\, t \tag{15}$$

D. Real-Time Task Architecture

The ESP32 Arduino core already runs on top of FreeRTOS, with `setup()` and `loop()` executing inside a kernel task [4, 5]. Rather than porting the hardware layers to the native ESP-IDF drivers, the firmware keeps the Arduino framework and the existing drivers and calls the FreeRTOS API directly. The application is organized as three tasks, summarized in Table 5.

Table 5 FreeRTOS Task Configuration

Task	Priority	Period	Role
Control	5 (highest)	20 ms	Owns the platform; solves IK and drives servos
Sequencer	3	20 ms	Generates interpolated poses
Reporter	1 (lowest)	5 s timeout	Logs control-loop execution time

Control task. The control task runs on a strict 20 ms period using `vTaskDelayUntil`, which wakes the task at fixed offsets from the previous wake time. Time spent on IK and I^2C therefore does not accumulate as drift, in contrast with a fixed delay after each iteration. On each period the task reads the newest target pose, solves IK, writes the six servo commands, and measures its own execution time. It has the highest priority so that jitter in the lower-priority tasks cannot delay the servo update. After startup it is the only code that touches the platform object. Because a single task performs all I^2C traffic, no mutex is needed on the bus.

Pose mailbox. The sequencer passes targets to the control task through a queue of length one that is written with overwrite semantics and read with a non-removing peek. The writer therefore never blocks, and the reader always acts on the most recent command rather than a backlog of stale ones. This is the usual pattern for "latest value" data between a producer and a periodic controller. The queue is wrapped in a small `PoseMailbox` class that owns the queue handle and exposes only `post` and `latest`.

Sequencer task. The sequencer runs a motion as a series of timed moves. Each move is computed with the quintic interpolation above and posted to the mailbox once per 20 ms using `vTaskDelayUntil`, replacing the blocking `setPose` calls and busy-wait timing loops of the original firmware. When the sequence ends, the final pose is neutral, which the control task then holds indefinitely, and the sequencer signals the reporter and deletes itself.

Reporter task. The reporter prints the control task's worst-case execution time every five seconds. It blocks on a task notification with a five-second timeout, so a single call serves as both its timer and its shutdown signal. When the sequencer notifies it at the end of a run, the reporter prints a final value and deletes itself. A task notification is used instead of a semaphore because it needs no separate kernel object.

Shared logging. The serial logger is the one resource used by several tasks. Early testing showed output from two tasks interleaving mid-line, so the logger now takes a mutex around each complete message.

E. Startup and Ownership Hazards

Several implementation details proved important during the port and are worth recording.

- *Initialization order.* The platform is initialized with the blocking `setPose` before any task is created, which is safe because no other task exists yet. The mailbox must be created before the first task, since the higher-priority control task preempts startup code as soon as it is created and reads the mailbox immediately.
- *Argument lifetime.* Tasks receive a single `void*`. Objects passed this way, such as the context structures that bundle the platform and mailbox pointers, must outlive the function that creates the tasks, so they are held as members of the application object rather than as locals.
- *Handle ordering.* The sequencer needs the reporter's task handle in order to notify it, so the reporter must be created first and its handle stored before the sequencer's context is filled. Copying the handle too early passes a null handle, which the kernel rejects with an assertion at the end of the run.
- *Shared state across files.* The worst-case execution time variable is declared `extern` in a shared header and defined once. Defining it as a `static` variable in the header gives every translation unit a private copy, so the reporter initially read zero while the control task updated a different instance.

F. Motion Sequences

The RTOS sequencer currently runs a six-DOF demonstration that moves each degree of freedom in both directions about neutral and returns to neutral between axes: heave (z of 162 mm and 190 mm around a 175 mm neutral height), surge and sway of ±45 mm, roll and pitch of ±0.2 rad, and yaw of ±0.3 rad.

Two continuous motion sequences were also implemented in the original blocking firmware, and are being ported to the sequencer.

Flat Precession. The platform centroid traces a horizontal circle of radius $A = 40\,\text{mm}$ at $f = 0.75\,\text{Hz}$ with no angular component:

$$x(t) = A\cos(2\pi f t), \qquad y(t) = A\sin(2\pi f t), \qquad z = 175\,\text{mm} \tag{16}$$

Pose updates were dispatched at 50 Hz using a non-blocking elapsed-time check in a busy-wait loop.

Angled Precession. Combines translational and angular circular motion to simulate a precessing gyroscope, with the platform leaning outward in the direction of travel:

$$x(t) = A_p\cos(\omega t), \quad y(t) = A_p\sin(\omega t), \quad \phi(t) = A_\theta\sin(\omega t), \quad \theta(t) = -A_\theta\cos(\omega t) \tag{17}$$

with $A_p = 30\,\text{mm}$, $A_\theta = 0.2\,\text{rad}$, $f = 0.25\,\text{Hz}$, updated at 75 Hz.

V. Results and Future Work

A. System Performance

The platform demonstrates all six degrees of freedom and executes smooth moves with zero velocity and acceleration at each endpoint. The dominant finding of the performance work was that the original numerical IK solver could not

sustain the intended 50 Hz update rate: at roughly 34 ms per solve it exceeded the 20 ms period, so moves requested as 400 ms took substantially longer than commanded. Replacing it with the closed-form solver reduced the solve time to about 71 µs, and requested move durations are now met to within the quantization of the control period.

The control task's measured worst-case execution time, which includes IK and the six servo writes over I^2C, was between about 3.2 ms and 3.7 ms across runs, against a 20 ms period, or under 19% of the available time. Since the IK solve accounts for under 0.1 ms of this, the remainder is consistent with the cost of the I^2C transfers, making the bus rather than the kinematics the dominant per-update cost. The reporter measures this value during the sequence, and the control task continues to hold neutral after the sequence completes.

B. Limitations

The numerical baseline's 0.1° angular resolution is no longer a limitation of the deployed solver, but the closed-form solver's root selection relies on the previous commanded angle to choose an elbow branch and does not yet normalize candidate angles into a fixed range. Each move begins and ends at rest, so multi-waypoint motions pause at every waypoint rather than blending through them. The two precession sequences have not yet been ported to the RTOS sequencer. The control task's IK-failure path writes to the logger, which can block on the logging mutex inside the 20 ms loop. The platform's park (sleep) routine still uses the blocking `setPose` call and so cannot safely be invoked while the tasks are running; it should become a command handled by the control task. Finally, only the control task's execution time is instrumented. Jitter in the wake-up time and the load imposed by the other tasks have not been measured. Additionally, the platform joint positions in `StewartConfig.h` currently use `SERVO_RAD` rather than the separate `PLATFORM_JOINT_RAD` constant, a known configuration discrepancy to be corrected.

C. Future Work

Several extensions are planned. First, the precession motions should be ported to the sequencer as time-sampled routines that sleep between updates, replacing their busy-wait loops. Second, logging should be removed from the control path, with errors reported through a counter or flag that the reporter prints. Third, a park command should be added that the control task handles, and the closed-form root selection should be hardened. Fourth, the mailbox makes it straightforward to add a serial or BLE command task as a second producer for real-time teleoperation. Fifth, mounting an IMU on the platform and closing the control loop against a reference orientation would enable active stabilization, using the existing fixed-period control task. Finally, sweeping the IK solver across a grid of poses would generate a reachability map useful for motion planning and workspace visualization.

D. Conclusion

This paper has presented a fully functional embedded Stewart Platform using commodity servo hardware and a compact C++ firmware stack. A closed-form inverse kinematics solver removed the main timing bottleneck of an initial numerical design, reducing the solve time from about 34 ms to about 71 µs and making a 50 Hz control rate achievable. The firmware was then restructured as three FreeRTOS tasks with a single-owner control task, a single-slot overwrite mailbox, and fixed-period scheduling, with a measured worst-case control-loop execution time of roughly 3.7 ms against a 20 ms period. The layered architecture makes the codebase straightforward to extend to new geometries, motion profiles, and sensing.

References

[1] Stewart, D., "A Platform with Six Degrees of Freedom," *Proceedings of the Institution of Mechanical Engineers*, Vol. 180, No. 1, 1965, pp. 371–386.

[2] NXP Semiconductors, "PCA9685: 16-channel, 12-bit PWM Fm+ I^2C-bus LED Controller," Product Data Sheet, Rev. 4, 2015.

[3] Merlet, J.-P., *Parallel Robots*, 2nd ed., Springer, Dordrecht, 2006.

[4] Barry, R., *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*, Real Time Engineers Ltd., 2016.

[5] Espressif Systems, "ESP-IDF Programming Guide: FreeRTOS (IDF)," Espressif Systems documentation.